

Force Tracker

Dossier complet pour relecture externe
2 août 2026

À quoi sert ce dossier

Michel, fondateur de Force Tracker, souhaite un **avis extérieur objectif** sur la fiabilité de son application. Deux documents partiels lui ont déjà été remis ; ils ont produit une analyse faussée, faute de contexte. Sa consigne : « *s'il n'a pas tous les éléments à sa disposition, il ne pourra pas analyser correctement le problème, il ne pourra pas faire un retour objectif* ».

Ce dossier **remplace** les deux précédents. Il est autonome : aucune connaissance préalable du projet n'est nécessaire.

Statut des informations

Tout chiffre de ce dossier est une **sortie de programme** : le code est exécuté dans un navigateur réel et interrogé. Rien n'est estimé ni recopié de mémoire. Quand une information n'a **pas** été vérifiée, c'est écrit explicitement.

Attention aux dates : plusieurs correctifs importants ont été livrés **dans la journée du 2 août**. Une observation faite le matin peut être exacte et périmée l'après-midi. Les numéros de version (ft-vNNN) permettent de dater chaque affirmation.

Sommaire

Partie	Contenu	Pourquoi c'est nécessaire
I	Le produit et sa philosophie	Sans l'intention, on juge mal les compromis
II	L'architecture technique	Contraintes réelles (pas de framework, pas de base de données)
III	Le modèle des exercices	Le cœur du problème soumis
IV	Milo, l'assistant conversationnel	Ce qu'il reçoit vraiment, mesuré
V	Les données de l'utilisateur	Stockage, synchronisation, restauration
VI	Le dispositif de vérification	12 familles de tests — ce qu'elles couvrent et ne couvrent pas
VII	La méthode de travail du projet	Règles, journal, catalogue de bugs
VIII	L'audit du 2 août : méthode et résultats	3 méthodes, dont une échouée
VIII bis	Analyse causale des 19 défauts	Données ? règles ? modèle ? cas isolé ?
VIII ter	Confiance, audit des règles, empreinte	82 % sans ambiguïté · 18 % dépendants de l'ordre
IX	Les limites connues, mesurées	La partie la plus importante
X	Les questions posées au relecteur	9 questions

PARTIE I — Le produit et sa philosophie

Force Tracker est une application web de suivi de musculation, utilisée sur téléphone. Elle est née le 17 juin 2026 et compte aujourd'hui environ **730 versions livrées**. Elle est utilisée par son auteur et une poignée de testeurs réels (Christophe, Tatiana, Emma, Eline) ; elle n'est pas encore ouverte au public.

Michel **n'est ni développeur ni programmeur**. Il conçoit le produit, teste, arbitre. Le code est écrit par un assistant IA sous sa direction. Cela explique deux choses : la gouvernance écrite très dense (partie VII), et le fait que les décisions produit soient toujours les siennes.

La phrase qui tient lieu de cap

« *Force Tracker n'est pas une intelligence artificielle. C'est une mémoire sportive intelligente.* »
« *Il ne te dit pas qui tu dois devenir, il se souvient de qui tu es devenu.* »

Ce cap a des conséquences techniques directes, qu'il faut avoir en tête pour juger les compromis : la **mémoire longue** est une fonctionnalité centrale et non un bonus ; la **perte de données** est traitée comme la faute la plus grave possible ; et l'assistant doit **observer avant de conseiller**.

Les 12 règles d'or du projet

Elles sont relues à chaque session de travail. Trois concernent directement le sujet soumis (n° 3, 10, 12).

#	Règle
1	Apps Script : toujours redéployer après un changement de code
2	Premium : ne jamais écraser la liste des accès
3	Zéro perte de séance — priorité n° 1 absolue
4	Ouverture instantanée à la salle, même hors ligne
5	Incrémenter le numéro de version à chaque déploiement
6	Avant toute opération risquée : sauvegarde et branche
7	Garder l'identité visuelle ; une chose à la fois, testée
8	Commit étiqueté avant, tag stable après, retour arrière en une ligne
9	Le bouton central de l'écran Séance est sensible : vérifier son positionnement
10	Michel n'est ni développeur ni programmeur : expliquer simplement, prévenir avant tout risque
11	À chaque fonctionnalité livrée : informer l'utilisateur (point rouge, aide, guide)
12	Tenir tous les fichiers de suivi à jour en temps réel

PARTIE II — L'architecture technique

Contrainte fondamentale : il n'y a ni framework, ni étape de compilation, ni base de données relationnelle. C'est du HTML, du CSS et du JavaScript servis tels quels. Cette contrainte est assumée (démarrage instantané, fonctionnement hors ligne) mais elle explique beaucoup de choix qui paraîtraient étranges dans un projet classique.

Fichier	Rôle	Lignes
index.html	Structure de toutes les pages (application mono-page)	2 690
style.css	Totalité du style, thèmes clair et sombre	1 360
constants.js	Le catalogue des exercices , les groupes, les nouveautés	480
state.js	L'objet d'état global, chargement et sauvegarde locale, calculs nutritionnels	824
app.js	Démarrage, nutrition, cardio, calories, panneau d'administration	3 333
screens.js	Navigaton, écran d'accueil, aides contextuelles	1 917

Fichier	Rôle	Lignes
log.js	Écran séance, classification des exercices, moteur des muscles	5 471
coach.js	Milo : construction du contexte, envoi, garde-fous	3 561
setup.js	Profil, progrès, graphiques, synchronisation et restauration	2 461
tracking.js	Cycle de force, badges, récupération, notifications	2 420
sw.js	Cache hors ligne (Service Worker), versionné à chaque livraison	267
Code.js	Serveur (Google Apps Script) : sauvegarde, IA, premium	2 298

Le serveur est un script Google Apps Script adossé à une feuille de calcul et à des « propriétés de script » (un magasin clé-valeur de **512 Ko au total, partagé par tous les comptes**). Ce plafond a déjà provoqué une panne réelle (partie IX). Les comptes y sont stockés compressés. L'hébergement de l'application est GitHub Pages ; le déploiement est automatique à chaque envoi de code.

PARTIE III — Le modèle des exercices — le cœur du sujet

Une fiche d'exercice contient exactement deux champs. Mesuré sur les 375 entrées du catalogue (337 noms uniques ; certains apparaissent dans deux groupes) :

n : le nom de l'exercice g : le groupe musculaire

Il n'existe aucun identifiant. Tout le reste est recalculé à la volée à partir du nom, à chaque affichage :

Information déduite	Mécanisme	Résultat possible
Muscles travaillés	69 règles (expressions régulières) parcourues dans l'ordre ; la première qui correspond gagne et donne des muscles <i>principaux</i> et <i>secondaires</i> .	17 muscles : abs, biceps, calves, forearms, front-delt, glutes, hamstrings, hip-flexors, lats, lower-back, obliques, pec, quads, rear-delt, side-delt, traps, triceps
Schéma de mouvement	Liste de mots-clés, même principe.	20 schémas (squat, charnière de hanche, poussée / tirage horizontal et vertical, gainage, porté, cardio...)
Matériel	Mots-clés dans le nom.	8 catégories (barre, poids libre, guidé, poids du corps, élastique, sangles, cardio, polyvalent)
Dépense calorique	Déduite du nombre et de la région des muscles trouvés.	MET 4 (isolation) · 5,5 (haut du corps) · 6,5 (bas du corps) · 8 (cardio, haltérophilie)
Rôle dans un programme	Déduit du schéma de mouvement.	« ancre » (mouvement principal) ou « accessoire »

Conséquence à garder en tête pour toute la suite

Le nom d'un exercice n'est pas une étiquette d'affichage : c'est à la fois **l'entrée de cinq calculs** et **la clé primaire de l'historique**. Ajouter un mot dans un nom modifie donc silencieusement des statistiques, et renommer un exercice casse le lien avec le passé si aucune migration n'est écrite.

Répartition du catalogue

Groupe (choisi à la main)	Exercices
Jambes	58
Dos	52
Épaules	47
Pectoraux	41
Fessiers	34
Triceps	25
Abdominaux	20
Full Body	17
Biceps	16
Lombaires	8
Mollets	8
Trapèzes	6
Avant-bras	5

Qualité du classement : **317 exercices sur 337 (94 %)** sont classés par une règle **précise** ; seulement **20 (6 %)** par une règle de rattrapage large. Moyenne de **1,66 muscle principal** et **1,96 muscle secondaire** par exercice.

PARTIE IV — Milo — l'assistant conversationnel

Milo est un modèle de langage (Claude) appelé via le serveur. Il n'a **aucune mémoire propre** : tout ce qu'il sait lui est envoyé à chaque message, dans un contexte reconstruit à la volée. **C'est donc la composition de ce contexte qui détermine son intelligence apparente.**

Composition mesurée du contexte

Mesure sur un profil réaliste : **57 954 caractères** répartis en 18 blocs.

Bloc	Taille	Part	Nature
TA PERSONNALITÉ	19 716	34 %	Instructions de comportement
PROFIL ATHLÈTE	11 292	19 %	Instructions + données
SE SOUVENIR DE LA PROCHAINE SÉANCE	9 031	15 %	Instructions (format de sortie)
NUTRITION	6 808	11 %	Instructions
COHÉRENCE AVANT RÉACTIVITÉ	1 808	3 %	Instructions
PERTINENCE AVANT DISPONIBILITÉ	1 796	3 %	Instructions
MÉMOIRE DURABLE	1 563	2 %	Instructions
MÉTHODE DE COACHING	1 333	2 %	Instructions
(10 autres blocs)	—	—	Mixte

Sur ces 57 954 caractères, environ 2 400 (4 %) sont des données sur la personne. Le reste est constitué d'instructions. Ce déséquilibre a été mesuré fin juillet ; une réduction des instructions a été envisagée puis **abandonnée sur décision de Michel** : « *tu me fais flipper là, parce que franchement Milo il est au top* ». Rien n'était cassé, le budget n'était pas saturé : retirer des règles qui fonctionnent aurait été un risque pur.

La mémoire de l'historique — point souvent mal compris

Milo n'est pas limité aux cinq dernières séances. Il reçoit deux blocs de nature différente :

Bloc	Contenu	Portée
Séances détaillées	Chaque série, chaque charge, chaque répétition	5 dernières
Parcours depuis l'inscription	Ancienneté, nombre total de séances, régularité hebdomadaire, plus longue coupure , volume cumulé, progression estimée par exercice	Tout l'historique

Sortie réelle, mesurée sur un profil de 91 séances comportant un arrêt de trois mois :

PARCOURS DEPUIS L'INSCRIPTION :

- Première séance enregistrée : 19 août 2025 (il y a 348 jours) · 91 séances au total
- Régularité : 1,8 séance par semaine en moyenne · **plus longue coupure : 91 jours (reprise le 13 juillet 2026)**
- Volume cumulé : 188 tonnes soulevées depuis le début
- Progression : Squat à la Barre : 90 -> 113 kg estimés (+25 %, 91 séances)

Coût : environ **650 caractères** (1 % du contexte), construit en 14 ms sur 366 séances. Ce bloc a été livré le **2 août à la mi-journée** (version ft-v727). **Avant cette date, la limitation aux cinq séances était réelle.**

PARTIE V — Les données de l'utilisateur

Structure d'une séance enregistrée

```
séance : { date, exs[], vol }  
exercice : { name, sets[], note }  
série   : { kg, reps, done, type, rm1 }
```

Point capital : les muscles ne sont pas enregistrés. Une séance ne stocke que le **nom** de l'exercice ; les muscles sont recalculés à chaque affichage. Corriger une règle aujourd'hui change donc rétroactivement ce que les séances d'il y a un an « ont travaillé ». Utile pour propager un correctif, gênant pour la reproductibilité des statistiques.

Stockage et synchronisation

Niveau	Support	Capacité / limite
Local (prioritaire)	Stockage du navigateur, 126 clés préfixées ft4_	Jusqu'à 1 500 séances ; quota du navigateur
Cloud	Propriétés du script Google, compte compressé	512 Ko partagés par tous les comptes
Sauvegarde	Fichiers JSON sur Google Drive, tâche nocturne quotidienne	Sans limite pratique

Le principe est **local d'abord** : on enregistre sur le téléphone avant toute synchronisation, et le réseau ne doit jamais bloquer ni faire perdre une donnée. À la restauration, l'application prend **la version la plus complète** entre le local et le cloud.

Le défaut de perte de données corrigé le 2 août

Trouvé en vérifiant une inquiétude de Michel sur la mémoire. Il est instructif parce qu'**aucune de ses quatre étapes n'est absurde** :

Étape	Comportement	Jugement isolé
1	Le stockage du téléphone sature : l'application réduit l'historique local à 50 séances et affiche « <i>tes séances restent sauvegardées dans le cloud</i> »	Raisonné (évite un plantage)
2	Au redémarrage, l'application ne relit que ces 50 séances	Cohérent
3	À la sauvegarde suivante, elle envoie ces 50 séances au serveur	Comportement normal
4	Le garde-fou serveur ne refusait que les envois vides : 50 séances remplaçaient 500, en silence	Destruction de données

Le message de l'étape 1 devenait **faux** à l'étape 4, et **rien ne traçait l'événement** — il était donc impossible de savoir s'il s'était produit. Corrigé (ft-v732) par trois barrières : un drapeau qui suspend l'envoi tant que la copie locale est incomplète · un garde-fou serveur qui refuse tout rétrécissement brutal · une trace remontée dans le tableau de santé, *parce qu'une alerte qui ne remonte nulle part ne sert à personne*.

PARTIE VI — Le dispositif de vérification

Douze familles de tests automatisés tournent avant chaque livraison, dans un navigateur réel (Chromium sans interface). Une famille rouge **bloque la livraison**. Le principe directeur : **chaque bug découvert devient un scénario de test permanent**.

Famille	Ce qu'elle protège	Volume
milo	Noyau dur : scénarios conversationnels critiques	10 scénarios
anneau	Affichage de la récupération, régions musculaires	110 tests
calendrier	Couleurs et agrégats du calendrier	10 tests
calendrier-milo	Ce que Milo sait du calendrier (jours, dates)	24 tests
questionnaire	Parcours d'inscription	18 tests
discussions	Contexte conversationnel, mémoire, coût du contexte	36 tests
dates	Interdit d'utiliser l'heure de Greenwich pour la date du jour	7 tests
donnees	Chaque donnée doit être classée face à Milo (transmise / exclue avec raison / trou connu)	garde-fou
calculs	Calculs isolés : calories, macros, récupération, 1RM	124 tests
parcours	Parcours utilisateur complets de bout en bout	128 tests
muscles	Classification musculaire sur tout le catalogue	117 tests
croises	Contradictions entre sources (créée le 2 août)	31 tests

Deux garde-fous méritent d'être signalés car ils sont inhabituels :

— **La famille « donnees »** lit toutes les données chargées par l'application et exige que chacune soit **classée** : transmise à Milo, exclue avec la raison écrite, ou trou connu. Une donnée non classée fait échouer la livraison. Elle a bloqué une livraison le 2 août encore. Motif de sa création : la même erreur (une donnée collectée mais jamais transmise) s'était produite **cinq fois**, et l'oubli est **silencieux** — il ne casse rien, il rend seulement les réponses un peu moins bonnes.

— **Le contrôle négatif systématique** : avant chaque livraison, les fichiers modifiés sont mis de côté et les tests relancés — ils **doivent** échouer. Un test qui reste vert sans le correctif ne prouve rien.

PARTIE VII — La méthode de travail du projet

Ce point est inclus parce qu'il fait partie de ce qui est soumis au jugement : la fiabilité vient autant de la méthode que du code.

Document	Rôle	Taille
CLAUDE.md	Page d'accueil du projet : règles d'or, architecture, journal récent	14 600 mots
docs/JOURNAL-ARCHIVE.md	Journal complet des ~730 versions, avec le <i>pourquoi</i> de chacune	108 500 mots
docs/REGLES-ARCHITECTURE.md	30 règles de conception , chacune née d'un incident réel	3 900 mots
CONSTITUTION-MILO.md	Principes de comportement envers la personne (éthique, sécurité)	5 900 mots
BUGS.md	Catalogue des bugs par FAMILLE (créé le 2 août)	3 400 mots
docs/REGLES-OR.md	Les 12 règles d'or en version longue	1 400 mots

Les règles de conception les plus pertinentes ici

Règle	Énoncé	Née de
R4	L'information doit descendre jusqu'à la donnée , jamais rester dans le texte	5 bugs où Milo raisonnait juste sans que ça atteigne l'application
R8	Un prompt ne compense jamais une donnée absente	Le prompt citait une source qu'on ne lui envoyait pas
R14	Un comportement copié d'un contexte à un autre peut devenir faux	Pré-remplissage de charges
R17	Chaque bug découvert devient un scénario de test permanent	Framework de tests
R23	Une fonctionnalité livrée sans entrée de journal devient invisible	Une fonctionnalité déclarée manquante alors qu'elle existait depuis 3 semaines
R28	Une limite non vérifiée devient une règle de conception silencieuse	Michel s'est bridé graphiquement pendant des semaines sur une limite inexistante
R29	Le droit de deviner dépend du coût de l'erreur	Erreur gratuite : devine. Erreur qui touche la personne : demande
R30	Un retrait volontaire doit être écrit, sinon il redevient un bug	Une suppression délibérée a été « réparée » trois mois plus tard

Le catalogue des bugs, rangé par famille

Le constat qui a motivé sa création : sur ~730 versions, **les mêmes cinq ou six bugs reviennent sous des déguisements différents**. Un bug isolé est une anecdote ; un bug qui revient douze fois est une propriété du système.

Famille de bug	Occurrences
Le « premier match gagnant » (règle générale masquant une règle précise)	au moins 12
L'information n'atteint jamais la donnée	11
Le temps et les fuseaux horaires	au moins 6
Deux sources qui se contredisent	14 (toutes trouvées le 2 août)
Le déploiement silencieux	3
Les seuils en marche d'escalier	3
La promesse écrite à l'utilisateur, et fausse	2
Les erreurs de méthode (mesure fausse, contrôle inopérant)	au moins 5

PARTIE VIII — L'audit du 2 août : méthode et résultats

Point de départ

Une testeuse tape « **Tirage horizontal** » dans la liste des exercices et obtient « **Aucun résultat** » — alors que ce terme est le nom exact d'une des familles de mouvement de l'application, qui y classe 33 exercices. Puis Michel formule la question de fond : « *il y a sûrement certains exercices qui ne sont pas dans la bonne catégorie* ».

Méthode A — comparaison à une base externe : ÉCHEC

Une base publique de 873 exercices avec leurs muscles a été téléchargée et confrontée au catalogue. **Résultat inutilisable** : elle classe le **développé couché** en « triceps » comme muscle principal ; son vocabulaire est plus grossier que le nôtre (« shoulders » au lieu de trois portions du deltoïde) ; et l'appariement automatique produisait des faux amis (« Curl Poignet Barre » apparié à « Barbell Curl »). Bilan : 28 « désaccords » dont environ 3 réels après lecture humaine.

Méthode B — audit linéaire des 69 règles

Vérifier les **règles** plutôt que les exercices divise le travail par cinq. Les 69 règles ont été lues une par une avec la liste des exercices que chacune attrape. **Résultat : 4 erreurs**, ensuite confirmées sur sources documentaires (NASM, BarBend, études EMG) : glute ham raise classé lombaires au lieu d'ischios · leg curl sur-attribué aux fessiers · planche latérale classée abdominaux au lieu d'obliques · adduction de cuisses classée fessiers au lieu d'adducteurs.

Méthode C — les croisements

Suggérée par Michel sur intuition : « *il faudrait croiser les données comme on a fait une fois en linéaire et en diagonale... je le sens, y'a un truc qui nous a échappé* ».

Le principe : l'application connaît **plusieurs choses indépendantes** sur un même exercice (groupe choisi à la main, muscles calculés, schéma de mouvement, terme de recherche anglais, fichier d'animation, catégorie de matériel). Chacune est plausible isolément et aucune ne provoque d'erreur.
On ne tient un bug que lorsque deux se contredisent.

Résultat : 14 défauts supplémentaires, soit 3,5 fois plus que la lecture linéaire. Parmi eux : 4 doublons (deux fiches pointant la même animation ou le même terme anglais) · un exercice de **poussée** classé en **tirage** (l'application croyait qu'on avait tiré alors qu'on avait poussé) · une règle **morte** qui faisait recevoir aux oiseaux et face pull le deltoïde *moyen* en muscle principal au lieu du *postérieur* (10 exercices) · deux exercices dont le nom dit « haltère » rangés en machine et en poids du corps.

Ces six croisements ont été transformés en **famille de tests permanente**. Elle a attrapé un 14■ défaut **dès son premier lancement** : le *Jefferson Curl*, une flexion vertébrale chargée, était classé comme un curl de biceps.

Second audit (soir) — la trouvabilité, déclenchée par le même retour

Michel revient sur le déclencheur : « *une adhérente n'a pas trouvé un exercice. Je ne veux pas corriger uniquement ce cas. Je veux comprendre si ce bug révèle une faiblesse de l'architecture.* » Une seconde mesure a donc été faite, sur la **trouvabilité** plutôt que sur la classification.

Protocole : pour chacun des 337 exercices, simuler une recherche par le **premier mot de son nom**, puis relever le nombre de résultats et la **position** de l'exercice cherché dans la liste.

Mesure	Avant correctif	Après correctif
Exercices apparaissant au-delà du 20 ^e résultat	73	22
Exercices au-delà du 30 ^e résultat	32	—

Mesure	Avant correctif	Après correctif
Recherche « pec deck » (un nom d'exercice unique)	45 résultats , l'exercice en dernière position	1
Recherche « svend »	45 résultats, en dernière position	1
Recherches « yates » et « meadows »	31 résultats chacune	1
Recherche « développé couché »	45 résultats dont 8 seulement contenaient ces mots	8, tous pertinents

La cause, et elle est structurelle

La recherche est un **FILTRE** (oui / non) dont le résultat est affiché dans l'ordre **ALPHABÉTIQUE**. Elle n'a **aucune notion de « à quel point ça correspond »**.

Tant que le filtre restait étroit, le défaut était invisible. En l'élargissant aux familles de mouvement pour réparer le cas de la testeuse, le bruit est devenu ingérable : **chaque élargissement du filtre aggrave le problème, faute d'un tri pour le compenser**. Le correctif d'un symptôme avait donc créé un défaut plus large que celui qu'il réparait.

Corrigé (ft-v733) par deux mesures : un **rang de pertinence** explicite (nom exact > commence par > contient > terme anglais > groupe > famille de mouvement), et un élargissement par famille qui ne se déclenche plus que sur le **libellé** d'une famille — jamais sur un mot-clé isolé, car ces mots-clés contiennent des noms d'exercices précis (« svend », « pec deck », « yates »).

Un piège dans le piège, qui mérite d'être signalé : une fois le tri écrit, il ne se voyait toujours pas — l'affichage **regroupe les résultats par matériel**, et ce regroupement imposait son propre ordre. L'exercice avait beau être classé premier, il repartait dans sa catégorie, affichée plus bas. *Un tri ne sert à rien s'il ne traverse pas la mise en forme.*

Trouvabilité : les portes d'entrée de chaque exercice

Mesure complémentaire : par combien de chemins **différents** un exercice peut-il être atteint (son nom, son groupe musculaire, son terme anglais, sa famille de mouvement, sa catégorie de matériel) ?

Portes d'entrée	Exercices
5 portes	225
4 portes	108
3 portes	4
2 portes ou moins	0

Aucun exercice n'est orphelin. Les manques restants sont des trous de **donnée** et non de conception : 82 exercices n'ont pas de terme anglais, 34 ont un matériel indéterminé.

Couverture réelle de chaque croisement

Croisement	Couverture	Limite
Aucune règle morte	337/337	—
Groupe / muscles	337/337	—
Schéma / muscles	337/337	—
Animation en double	282/337 (83 %)	55 exercices n'ont pas d'animation
Terme anglais en double	255/337 (75 %)	82 exercices n'ont pas de terme anglais
Matériel / nom	143/337 (42 %)	On ne juge que si un seul matériel figure dans le nom

PARTIE VIII bis — Analyse causale des 19 défauts

Michel demande explicitement de ne pas s'arrêter à la liste des défauts : « *je veux identifier les causes. Est-ce un problème de données ? une limite des règles ? une limite du modèle ? un cas isolé ? une famille complète d'erreurs potentielles ?* » Voici la répartition, défaut par défaut.

Cause	Défauts	Exemples	Portée
Limite des règles (ordre de parcours)	9	Le « premier match gagnant » : une règle générale placée avant une règle précise la rend inatteignable. Oiseaux classés deltoïde moyen, Jefferson Curl classé curl de biceps.	Famille entière — au moins 12 occurrences historiques
Limite du modèle (tout se déduit du nom)	6	Ajouter une traduction dans un nom change sa catégorie de matériel. Deux fiches ne peuvent pas porter le même nom. Renommer casse le lien avec l'historique.	Structurelle — ne se résout pas par des correctifs
Limite du modèle (recherche sans pertinence)	1	Un filtre oui/non affiché par ordre alphabétique : chaque élargissement aggrave le bruit.	Famille entière — toute requête précise dont le mot figure dans une liste de mots-clés
Problème de données	3	Doublons (deux fiches pour le même exercice), animation associée au mauvais exercice.	Ponctuel, mais invisible sans croisement
Cas isolé	0	—	Aucun des défauts trouvés n'était réellement isolé.

Le résultat le plus important de cette analyse

Aucun des 19 défauts n'était un cas isolé. Chacun appartenait à une famille — c'est-à-dire que d'autres exercices étaient touchés par le même mécanisme, sans qu'on le sache. C'est ce qui justifie de corriger les causes plutôt que les cas.

Et deux défauts sur trois relèvent de la **même racine** : le nom de l'exercice est à la fois l'étiquette affichée, l'entrée de cinq calculs, et la clé primaire de l'historique. **Une seule chaîne de caractères porte trois responsabilités incompatibles.**

Chronologie de la journée du 2 août

Sept versions livrées en une journée. Elle est donnée parce qu'elle explique la plupart des malentendus possibles : une observation du matin peut être exacte et périmée le soir.

Version	Ce qui a changé
ft-v727	Mémoire longue : Milo reçoit tout le parcours depuis l'inscription, et plus seulement 5 séances
ft-v728	Recherche par famille de mouvement (correctif du cas de la testeuse) — <i>qui a introduit le défaut de pertinence</i>
ft-v729	Les deux noms (français et anglais) sur les exercices les plus répandus
ft-v730	Les croisements : 14 défauts invisibles trouvés, 12 ^e famille de tests créée, fichier des bugs créé
ft-v731	Couverture des croisements mesurée : deux trous comblés, limites restantes écrites
ft-v732	Zéro perte : le chemin par lequel l'historique cloud pouvait être écrasé
ft-v733	Rang de pertinence de la recherche (le défaut décrit en partie VIII)

PARTIE VIII ter — Indicateur de confiance et audit des règles

Cette partie répond à une demande du relecteur : « *peut-on mesurer la fiabilité du catalogue ? auditer les règles elles-mêmes ? détecter automatiquement une régression après une modification ?* » Les trois ont été mis en place et mesurés.

L'indicateur de confiance

Définition retenue, volontairement stricte : un exercice est classé **sans ambiguïté** si **toutes** les règles qui lui correspondent donnent le même résultat ; il est **fragile** si plusieurs règles lui correspondent en donnant des muscles **différents** — auquel cas sa justesse ne tient qu'à l'**ordre** des règles.

Mesure	Exercices	Part
Classés sans ambiguïté (indépendants de l'ordre)	277	82 %
Classés de façon fragile (le résultat dépend de l'ordre)	60	18 %
Sans aucun classement	0	0 %

Lecture indispensable de ce 18 %

Ce n'est **pas un taux d'erreur**. Les 60 exercices fragiles sont **corrects aujourd'hui** — le développé couché sort bien en pectoraux et non en épaules. C'est une **surface de risque** : ces 60 basculent silencieusement si une règle est insérée au mauvais endroit. C'est précisément ce qui est arrivé aux oiseaux (10 exercices) et aux élévations latérales.

L'audit des règles : le masquage partiel

Le dispositif détectait les règles **totale**ment mortes. Il ne voyait pas les règles **partiellement** masquées : celles qui capturent une partie de ce qui leur revient, le reste étant pris par une règle placée avant, avec un classement **différent**.

Constat	Valeur
Règles totale	0
Règles partiellement masquées	19 sur 69
La plus touchée : tirage pulldown row	capture 9 exercices, 19 lui échappent avec un autre classement
Deuxième : developpe shoulder press	capture 4 exercices, 16 lui échappent

Dans la plupart des cas le masquage est **voulu** (une règle précise doit passer avant une règle large). Ce qui manquait, c'est de le **voir**.

La détection automatique de régression

Une **empreinte** du catalogue a été créée : pour chacun des 337 exercices, ses muscles principaux et secondaires, son schéma de mouvement, sa catégorie de matériel et sa dépense calorique. Le test la compare à chaque version et **nomme** tout exercice qui bouge. Quand le changement est voulu, on régénère l'empreinte et l'historique du code montre exactement quels exercices ont été affectés.

Vérifié en cassant volontairement : en retirant la règle du deltoïde postérieur, le test échoue et affiche Machine Oiseau -> muscles vides. Jusqu'ici, modifier une règle ne montrait **jamais** qui d'autre était touché.

PARTIE IX — Les limites connues — la partie la plus importante

Limite de fond : un croisement ne détecte qu'une CONTRADICTION.

Si deux sources sont fausses **de la même façon**, le contrôle se tait. Un exercice dont le groupe, les muscles et le schéma seraient tous les trois cohérents **et tous les trois faux** reste totalement invisible.

Seule une vérification **externe** couvre cette zone — et elle n'a porté que sur **5 exercices sur 337**.

Limites du modèle des exercices

— **Aucun identifiant**. Le nom est la clé primaire de l'historique : renommer impose une table de migration (utilisée trois fois le 2 août).

— **Ce qu'on ne peut pas modéliser**, faute d'endroit où le mettre : unilatéral ou bilatéral, côté travaillé, amplitude, tempo, variantes d'un même mouvement, pondération continue des muscles (la hiérarchie est binaire).

— **Les statistiques passées ne sont pas figées** : elles se recalculent, donc elles changent quand une règle est corrigée.

— **Les tables d'attente des croisements sont écrites à la main** et n'ont été validées par personne. Une table trop permissive rend le contrôle décoratif : il passe au vert sans rien garantir.

Limites du raisonnement de Milo sur l'historique

Il reçoit la coupure, mais **mal cadrée**. Trois défauts mesurés :

#	Défaut mesuré	Conséquence
1	Aucune distinction entre une coupure passée et une coupure qui vient de se terminer : la donnée est présentée comme une statistique historique.	Le modèle doit <i>inférer</i> qu'il s'agit d'une reprise. Rien ne le lui dit.
2	Le signal explicite (« revient après une pause ») ne se déclenche que si la dernière séance date de plus de 14 jours.	Actif pendant l'absence, muet au retour — le seul moment qui compte.
3	La régularité est une moyenne sur toute la période, coupure comprise.	Mesuré : quelqu'un qui s'entraîne 3 fois par semaine depuis sa reprise est décrit comme « 0,6 séance par semaine ». Exact, et faux comme information.

Formulé autrement : l'application transmet des **chiffres** ; il lui manque de transmettre un **état** (« en reprise depuis 3 semaines après 3 mois d'arrêt, rythme actuel 3 fois par semaine, en hausse »). C'est calculable sur les données existantes.

Autres angles morts

Angle mort	Nature
Les exercices créés par l'utilisateur n'ont aucun schéma de mouvement	trou connu
Les programmes enregistrés et les temps de repos par exercice ne sont pas transmis à Milo	trous connus
Un exercice unilatéral compte comme un bilatéral dans le volume	fausse les statistiques
Le stockage cloud (512 Ko partagés) a déjà provoqué une panne de 2 jours ; contourné par compression, pas résolu	risque structurel
Milo est évalué par Michel sur un modèle haut de gamme ; les utilisateurs ont un modèle plus léger	biais d'évaluation

PARTIE X — Les questions posées au relecteur

Michel formule ainsi son objectif : « *je ne cherche pas la perfection absolue. Je souhaite que 95 à 99 % du catalogue soit fiable. Je préfère consacrer du temps maintenant plutôt que de construire des analyses ou de l'intelligence artificielle sur des données imparfaites.* »

Q1. Quels croisements manquent ?

Six ont été retenus. Pistes non explorées : le **nom du fichier d'animation** comparé au nom de l'exercice (c'est ce qui a révélé un doublon par hasard) ; le **groupe** comparé au **schéma de mouvement** ; la **cohérence interne des règles** entre elles.

Q2. L'architecture « tout se déduit du nom » est-elle le vrai problème ?

Six des dix-huit défauts en découlent directement. Faut-il la conserver en la contrôlant mieux, ou basculer vers des **attributs explicites** par exercice ? Contrainte : 337 fiches à renseigner et un historique indexé par nom.

Q3. Comment vérifier l'anatomie des 337 sans 337 recherches ?

Vérifier les 69 règles divise le travail par cinq, mais suppose que chaque exercice tombe sur la bonne règle — ce qui est justement le bug le plus fréquent. Existe-t-il une source de référence réellement fiable sur la hiérarchie principal / secondaire ?

Q4. Les tables d'attente sont-elles trop permissives ?

Elles définissent ce qu'un groupe ou un schéma « a le droit » d'avoir comme muscles, et n'ont été validées par personne. Comment calibrer cela honnêtement, sans rendre le contrôle décoratif ?

Q5. Que faire quand les sources se contredisent ?

Le **développé couché** est donné « pectoraux » par la quasi-totalité des sources et « triceps » par la base consultée. Le **pull-over** est donné tantôt dorsaux, tantôt pectoraux. Quelle règle d'arbitrage, sachant que l'application **doit** trancher pour colorier une silhouette anatomique ?

Q6. Y a-t-il un angle mort de méthode ?

Le projet tient la liste de ses propres erreurs de méthode : un contrôle négatif à zéro échec parce que le test *plantait* au lieu d'échouer ; une vérification sur 9 archétypes présentée comme une vérification du catalogue ; l'affirmation qu'une fonctionnalité manquait alors qu'elle existait depuis trois semaines. **Quel biais de ce type est encore à l'œuvre ici ?**

Q7. Faut-il transmettre un état narratif plutôt que des statistiques ?

Reprise, stagnation, montée en charge, changement de fréquence, retour de blessure. Quels états seraient réellement utiles à un coach, et sur quels seuils les déclencher **sans produire de faux signaux** ? (Le projet a déjà une règle : ne jamais sur-réagir au bruit — 84,8 kg puis 84,5 kg n'est pas une tendance.)

Q8. L'absence d'identifiant est-elle une dette à rembourser maintenant ?

Argument pour attendre : 337 fiches à renseigner, historique indexé par nom, aucun utilisateur public à ce jour. Argument pour agir : chaque mois ajoute des données indexées par une clé instable, et les fonctionnalités prévues (remplacement intelligent d'exercices, programmes automatiques, mode coach) en dépendent toutes.

Q9. Les statistiques recalculées : défaut ou propriété ?

Les muscles ne sont pas figés dans l'historique. Corriger une règle change donc le passé. Faut-il figer les muscles au moment de l'enregistrement (statistiques reproductibles mais erreurs gravées), ou conserver le recalcul (correctifs propagés mais passé mouvant) ?

Q10. Un correctif qui aggrave le défaut qu'il répare : comment l'éviter ?

Le cas est documenté dans ce dossier : élargir la recherche pour aider une testeuse a rendu les recherches précises inutilisables, et personne ne l'a vu pendant deux versions parce que les tests vérifiaient le **nombre** de

résultats et jamais leur **ordre**. Quelle classe de test aurait attrapé cela ? Plus généralement : **comment teste-t-on la pertinence d'un classement**, par opposition à son exactitude ?

Ce qui serait le plus utile en retour

Une critique de la **démarche** plutôt qu'une liste de corrections anatomiques. Concrètement : quels croisements ajouter, quelles hypothèses de la méthode sont fragiles, et surtout — **quelle catégorie d'erreur ce dispositif est-il structurellement incapable de voir ?**

Comment réfuter ce dossier : les mesures sont des sorties de programme, pas des opinions. Elles se contestent en pointant une erreur de **protocole de mesure**. À titre d'exemple, une mesure de ce dossier a déjà été invalidée puis refaite : un test cherchait le mot « reprise » dans le contexte et le trouvait toujours — parce que ce mot figure dans les *instructions* du modèle et non dans les *données*. Faux positif. C'est exactement le type d'erreur qu'un regard extérieur peut attraper.

Dossier généré automatiquement depuis le code source de l'application le 2 août 2026. Les chiffres (337 exercices, 69 règles, 17 muscles, 20 schémas, 12 familles de tests, tailles de fichiers, composition du contexte) sont lus dans l'application, pas saisis à la main : ce dossier est régénérable et ne peut pas se périmer en silence.